INFORME DEL TERCER PARCIAL

Cristian David Muñoz Hernández
0182410104

Universidad de la Guajira. Facultad de ingeniería
Ingeniería De Software I: **Grupo B1 - 273365**
Ing. Robert Damian Quintero Laverde

Especificaciones

Mis parámetros

A	B	C	D	E	F	PMAX	L1	L2	RR	RN	V	S	CAMBIO
4	1	0	1	0	4	50	3	10	10%	45%	15%	$2000	2

Historias de usuario

Historia de Usuario 1 (Reglas R1 y R2)

Soy cajero de EnvíaYa. Quiero que el sistema valide el peso, la zona y el día ingresado para evitar procesar envíos con datos erróneos o fuera de las políticas de la empresa.

Criterios de Aceptación:

- Dado que el cajero ingresa un peso de envío en el sistema, Cuando el peso es menor o igual a 0, o mayor al límite permitido de 50 kg (PMAX), entonces el sistema debe lanzar el error "peso inválido".

- Dado que el cajero ingresa los datos de destino y fecha, Cuando la zona ingresada no es "urbana", "rural" o "nacional", o el día de la semana no está en el rango de 1 a 7, entonces el sistema debe lanzar un error impidiendo el cálculo.

Historia de Usuario 2 (Reglas R3, R4 y R5)

Soy un cliente recurrente de EnvíaYa y quiero que mi tarifa base y recargos se calculen correctamente según el peso, destino y día de la semana para asegurar que se me cobre el precio justo.

Criterios de Aceptación:

- Dado un envío válido, Cuando el peso es estrictamente menor a 3 kg (L1 no incluido), entonces la tarifa base asignada es de $6.000.

- Dado un envío válido, Cuando el peso es mayor o igual a 3 kg y menor o igual a 10 kg (L2), entonces la tarifa base asignada es de $9.000.

- Dado un envío pesado, cuando el peso es mayor a 10 kg, entonces la tarifa es de $9.000 más un adicional de $800 por cada kg que sobrepase los 10 kg.

- Dado que se ha calculado la tarifa base, Cuando el destino es rural se suma un recargo del 10% (RR), y si es nacional se suma un 45% (RN), Entonces este valor se acumula a la tarifa base (la zona urbana no tiene recargo).

- Dado un envío a realizarse en fin de semana (día 6 o 7), Cuando se calcula el recargo de $2.000 (S), entonces este recargo debe sumarse estrictamente antes de aplicar cualquier descuento promocional o de cliente.

Historia de Usuario 3 (Reglas R6, R7 y R8)

Aplicación de descuentos, cupones y límites de precio (Reglas R6, R7 y R8). Como cliente de EnvíaYa Quiero que se apliquen correctamente los beneficios de fidelidad, cupones y políticas de cobro mínimo para obtener el precio total final a pagar con todos mis beneficios.
Criterios de Aceptación:

- Dado que el paquete ya tiene su tarifa con recargos de zona y fin de semana aplicados, Cuando el cliente tiene categoría frecuente (5%) o VIP (15%) y además presenta el cupón válido "ENVIO10" (10%), Entonces se aplican ambos descuentos de manera acumulativa, calculando primero el del tipo de cliente y luego el del cupón.

- Dado que un cliente intenta usar un cupón promocional, Cuando el texto del cupón ingresado no existe o es incorrecto, entonces el sistema lo ignora por completo y continúa con el cálculo normal sin lanzar error.

- Dado que se ha calculado el costo final con todos los descuentos, Cuando el monto resultante es menor a $5.000, Entonces el sistema ajusta el costo para cobrar una tarifa mínima de $5.000.

- Dado un precio final definitivo, Cuando el valor no es un múltiplo exacto de 100, entonces el resultado se redondea al múltiplo de $100 más cercano, promediando los $50 hacia arriba.

Matriz de trazabilidad

Matriz de Trazabilidad (Versión Inicial)

Requisito ID	**Descripción del Requisito (con parámetros propios) Parcial 3 - Ingenieria de Software.pdf**	**Historia de Usuario**	**Casos de Prueba (ID)**
R1	El peso debe cumplir 0 < peso ≤ 50 kg (PMAX). Si no, se lanza el error "peso inválido".	HU01: Validación de parámetros de entrada	*CP04, CP05, CP06, CP09*
R2	Si la zona no es urbana, rural o nacional, o el día no	HU01: Validación de	*CP20*

	está entre 1 y 7, se lanza un error.	parámetros de entrada	
R3	Tarifa base. Tramo 1 (peso < 3 kg): $6.000. Tramo 2 (hasta 10 kg, inclusive): $9.000. Más de 10 kg: $9.000 + $800 por cada kg adicional sobre 10 kg.	HU02: Cálculo de tarifa base y recargos	*CP01, CP07, CP08, CP10, CP11, CP12, CP13, CP15, CP17*
R4	Recargo por zona. Rural: +10% (RR). Nacional: +45% (RN). Urbana: sin recargo.	HU02: Cálculo de tarifa base y recargos	*CP02, CP03, CP16, CP18, CP19*
R5	Fin de semana (días 6 y 7): +$2.000 (S). Se suma **antes** de los descuentos.	HU02: Cálculo de tarifa base y recargos	*CP02, CP03, CP11, CP12, CP16, CP19*

R6	Descuento de cliente. Frecuente: 5%. VIP: 15% (V). Cupón ENVIO10: 10%. Si hay los dos (cliente y cupón), se aplican **ambos** (primero cliente, luego cupón).	HU03: Aplicación de descuentos, cupones y límites	*CP02, CP09, CP16*
R7	Un cupón que no existe se **ignora**.	HU03: Aplicación de descuentos, cupones y límites	*CP14*
R8	El costo nunca es menor que $5.000. El resultado se redondea al múltiplo de $100 más cercano; los $50 redondean hacia arriba.	HU03: Aplicación de descuentos, cupones y límites	*CP10, CP17*

Plan de pruebas

Alcance

El objetivo es verificar y validar la lógica de cálculo de la función calc en el módulo envios.py del sistema de EnvíaYa. Se evaluará el cálculo exacto de tarifas base, recargos por zona rural/nacional, descuentos de fidelidad/cupones y políticas de cobro mínimo y redondeo, garantizando el cumplimiento de las reglas de negocio personalizadas R1 a R8.

Niveles de Prueba

Cada nivel aborda una dimensión específica del sistema EnvíaYa:

- Pruebas Unitarias: Se aplicarán directamente a la función calc (y a los métodos individuales extraídos durante la refactorización) para verificar cálculos matemáticos precisos, como el recargo rural del 10% o el descuento acumulativo de cliente VIP y cupón.

- Pruebas de Integración: Entrarán en juego en la Parte 6, comprobando que las nuevas clases del patrón Estrategia (ej. estrategia de descuentos o zonas) interactúen correctamente con la función principal sin alterar el resultado.

- Pruebas de Sistema: Validará el flujo completo a través de la función calc, asegurando que la combinación de múltiples variables (peso, zona, día, cliente, cupón) produce el costo final esperado, incluyendo los límites de tarifa mínima ($5.000) y el redondeo.

- Pruebas de Aceptación: Se cruzará contra los Criterios de Aceptación de las historias de usuario HU01, HU02 y HU03, simulando escenarios reales de cajeros y clientes para confirmar que el software cumple su propósito comercial.

Estrategia, Ambiente y Roles

- Estrategia: Enfoque híbrido. Primero, un diseño de pruebas de Caja Negra (valores límite, particiones de equivalencia, tabla de decisión). Luego, un análisis de Caja Blanca para garantizar la cobertura estructural. Todo se automatizará usando pruebas de regresión.

- Ambiente: Entorno de desarrollo local, Python 3.x, suite de pruebas pytest (junto a pytest-cov para métricas de cobertura), y Git/GitHub para la gestión de la configuración.

- Roles: Asumirás un perfil multifuncional (Equipo de Calidad y Mantenimiento): diseñarás los casos de prueba (Tester), reportarás las fallas (Analista QA) y modificarás el código heredado (Desarrollador).

Riesgos y Mitigación

1. Riesgo: Romper la funcionalidad base durante la aplicación del patrón Estrategia o la eliminación del parámetro flag.
    - Mitigación: Automatizar toda la suite de pruebas (mínimo 20 casos) contra el código defectuoso heredado y congelar esa batería de tests. Ningún refactor se aprobará si la regresión falla.

2. Riesgo: Conflictos de integración en el control de versiones al resolver múltiples defectos.
    - Mitigación: Estricta adherencia a la estrategia de ramificación. Cada defecto se corregirá en una rama aislada (fix/DEF-X) y se integrará individualmente al tronco principal garantizando transiciones seguras de código.

Criterios de Entrada y Salida

- Criterios de Entrada: Parámetros base calculados (PMAX=50, L1=3, L2=10, RR=10%, RN=45%, V=15%, S=$2000), especificación R1-R8 aprobada, y repositorio inicializado en la versión v1.0.0.

- Criterios de Salida: 100% de los casos de prueba automatizados en ejecución verde, defectos detectados corregidos, deuda técnica mitigada,

patrón Estrategia implementado y cobertura de código de sentencias y ramas verificadas.

Diseño de casos de caja negra

Particiones de equivalencia

Variable	Clases Válidas (Rango / Valor)	Clases Inválidas
peso (numérico)	• CV1 (Tramo 1): 0 < peso < 3 • CV2 (Tramo 2): 3 <= peso <= 10 • CV3 (Tramo pesado): 10 < peso <= 50	• CI1 (Cero o negativo): peso <= 0 • CI2 (Excede máximo): peso > 50 • CI3 (Tipo de dato inválido): No numérico / null / vacío
zona (texto)	• CV4: "urbana" • CV5: "rural" • CV6: "nacional"	• CI4 (Valor no listado): Cualquier otro texto (ej. "internacional") • CI5 (Formato/Vacío): Cadena vacía "", null o tipo no texto
cliente (texto)	• CV7: "normal" • CV8: "frecuente" • CV9: "vip"	• CI6 (Valor no listado): Cualquier otro texto (ej. "corporativo") • CI7 (Formato/Vacío): Cadena vacía "", null o tipo no texto

dia (entero)	• CV10 (Semana): 1 <= dia <= 5 (Enteros) • CV11 (Fin de semana): 6 <= dia <= 7 (Enteros)	• CI8 (Menor al límite): dia < 1 (ej. 0, -1) • CI9 (Mayor al límite): dia > 7 (ej. 8) • CI10 (Tipo inválido): Número decimal (ej. 3.5), texto o null
cupon (texto)	• CV12 (Con descuento): "ENVIO10" • CV13 (Sin descuento - Ignorado por RN): Cualquier otro texto distinto a "ENVIO10" (ej. "PROMO50") o cadena vacía ""	• CI11 (Tipo de dato inválido): Tipo no texto (ej. numérico sin comillas, objeto/array) si el contrato de la función exige string

Valores límite

Frontera Evaluada	Valor de Prueba (kg)	Clase/Comportamiento Esperado
Alrededor de 0 (Límite inferior general)	-0.1	Inválido (Error)
	0.0	Inválido (Error)
	0.1	Válido (Tramo 1)
Alrededor de L1 = 3 (Transición Tramo 1 a Tramo 2)	2.9	Válido (Tramo 1: peso < 3)
	3.0	Válido (Tramo 2: peso >= 3)
	3.1	Válido (Tramo 2: peso >= 3)

Alrededor de L2 = 10 (Transición Tramo 2 a Tramo Pesado)	9.9	Válido (Tramo 2: peso <= 10)
	10.0	Válido (Tramo 2: peso <= 10)
	10.1	Válido (Tramo Pesado: peso > 10)
Alrededor de PMAX = 50 (Límite superior general)	49.9	Válido (Tramo Pesado: peso <= 50)
	50.0	Válido (Tramo Pesado: peso <= 50)
	50.1	Inválido (Error: peso > 50)

Tabla de decisión

Elemento	Condición / Acción	R1	R2	R3	R4	R5	R6	R7	R8
Condición	C1: ¿Es fin de semana (días 6 o 7)?	V	V	V	V	F	F	F	F
Condición	C2: ¿Tiene descuento de cliente (Frecuente o VIP)?	V	V	F	F	V	V	F	F
Condición	C3: ¿Tiene cupón válido (ENVIO10)?	V	F	V	F	V	F	V	F
Acción	A1: Sumar recargo de $2.000 a la base actual	1	1	1	1	-	-	-	-

Acción	A2: Aplicar descuento porcentual del cliente (5% o 15%) sobre el subtotal	2	2	-	-	1	1	-	-
Acción	A3: Aplicar descuento porcentual del cupón (10%) sobre el nuevo subtotal	3	-	2	-	2	-	1	-
Acción	A4: No aplicar ningún descuento ni recargo extra	-	-	-	-	-	-	-	X

Matriz de casos de prueba

ID	Requisitos	Entradas (Peso, Zona, Cliente, Día, Cupón)	Resultado Esperado	Técnica
CP01	Tramo 1, zona urbana, cliente normal, día laborable, sin cupón	2.5 kg, urbana, normal, 3, –	$6 000	Partición
CP02	Tramo 2, zona rural, cliente frecuente, fin de	5 kg, rural, frecuente, 6, ENVIO10	$10 200	Decisión

	semana, con cupón			
CP03	Peso > 10, zona nacional, cliente VIP, fin de semana, sin cupón	12 kg, nacional, vip, 7, –	$14 800	Decisión
CP04	Peso = 0 (inválido)	0 kg, urbana, normal, 2, –	Error	Límite
CP05	Peso negativo (inválido)	-1 kg, rural, frecuente, 5, –	Error	Límite
CP06	Peso > PMAX (inválido)	51 kg, nacional, vip, 4, –	Error	Límite
CP07	Límite inferior tramo 2 (peso = 3)	3 kg, urbana, normal, 1, –	$9 000	Límite
CP08	Límite superior tramo 2 (peso = 10)	10 kg, rural, normal, 5, –	$9 900	Límite
CP09	Peso = PMAX, zona nacional,	50 kg, nacional, frecuente, 2, ENVIO10	$50 800	Límite

	cliente frecuente, cupón válido			
CP10	Peso justo bajo L1, cliente VIP	2.9 kg, urbana, vip, 4, –	$5 100	Límite
CP11	Peso justo arriba L1, zona rural, fin de semana, cliente frecuente	3.1 kg, rural, frecuente, 6, –	$11 300	Límite
CP12	Peso justo bajo L2, zona nacional, fin de semana	9.9 kg, nacional, normal, 7, –	$15 100	Límite
CP13	Peso justo arriba L2, zona urbana	10.1 kg, urbana, normal, 1, –	$9 100	Límite
CP14	Peso pesado, cupón inválido (ignorado)	25 kg, urbana, normal, 3, XYZ	$21 000	Partición
CP15	Tramo 2, zona rural, sin descuentos	8 kg, rural, normal, 2, –	$9 900	Partición

CP16	Peso pesado, zona nacional, fin de semana, cliente frecuente, cupón válido	15 kg, nacional, frecuente, 6, ENVIO10	$17 800	Decisión
CP17	Tramo 2, cliente VIP, sin cupón	4 kg, urbana, vip, 5, –	$7 700	Partición
CP18	Tramo 2, zona rural, cliente frecuente, sin cupón	7 kg, rural, frecuente, 4, –	$9 400	Decisión
CP19	Peso pesado, zona nacional, fin de semana, sin cupón	20 kg, nacional, normal, 6, –	$26 700	Decisión
CP20	Día fuera de rango (inválido)	9 kg, urbana, normal, 8, –	Error	Partición

Cálculo Paso a Paso (5 casos complejos)

CP02

1. Base (peso 5 kg, tramo 2): $9 000
2. Zona Rural (+10 %): $9 000 × 1.10 = $9 900
3. Fin de semana (+$2 000): $9 900 + $2 000 = $11 900 *(Regla E=0: se suma antes de los descuentos)*

4. Descuento Cliente (Frecuente 5 %): $11 900 − (5 % × $11 900) = $11 305

5. Descuento Cupón (10 %): $11 305 − (10 % × $11 305) = $10 174.5
   (Regla B=1: se aplican ambos de forma acumulativa)

6. Redondeo al 100 más cercano: $10 174.5 → $10 200

CP03

1. Base (peso 12 kg): $9 000 + $800 × (12-10) = $10 600

2. Zona Nacional (+45 %): $10 600 × 1.45 = $15 370

3. Fin de semana (+$2 000): $15 370 + $2 000 = $17 370

4. Descuento Cliente (VIP 15 %): $17 370 − (15 % × $17 370) = $14 764.5

5. Sin cupón → subtotal $14 764.5

6. Redondeo: $14 764.5 → $14 800

CP09

1. Base (peso 50 kg): $9 000 + $800 × (50-10) = $41 000

2. Zona Nacional (+45 %): $41 000 × 1.45 = $59 450

3. Fin de semana (no) → subtotal $59 450

4. Descuento Cliente (Frecuente 5 %): $59 450 − (5 % × $59 450) = $56 477.5

5. Descuento Cupón (10 %): $56 477.5 − (10 % × $56 477.5) = $50 829.75

6. Redondeo: $50 829.75 → $50 800

CP16

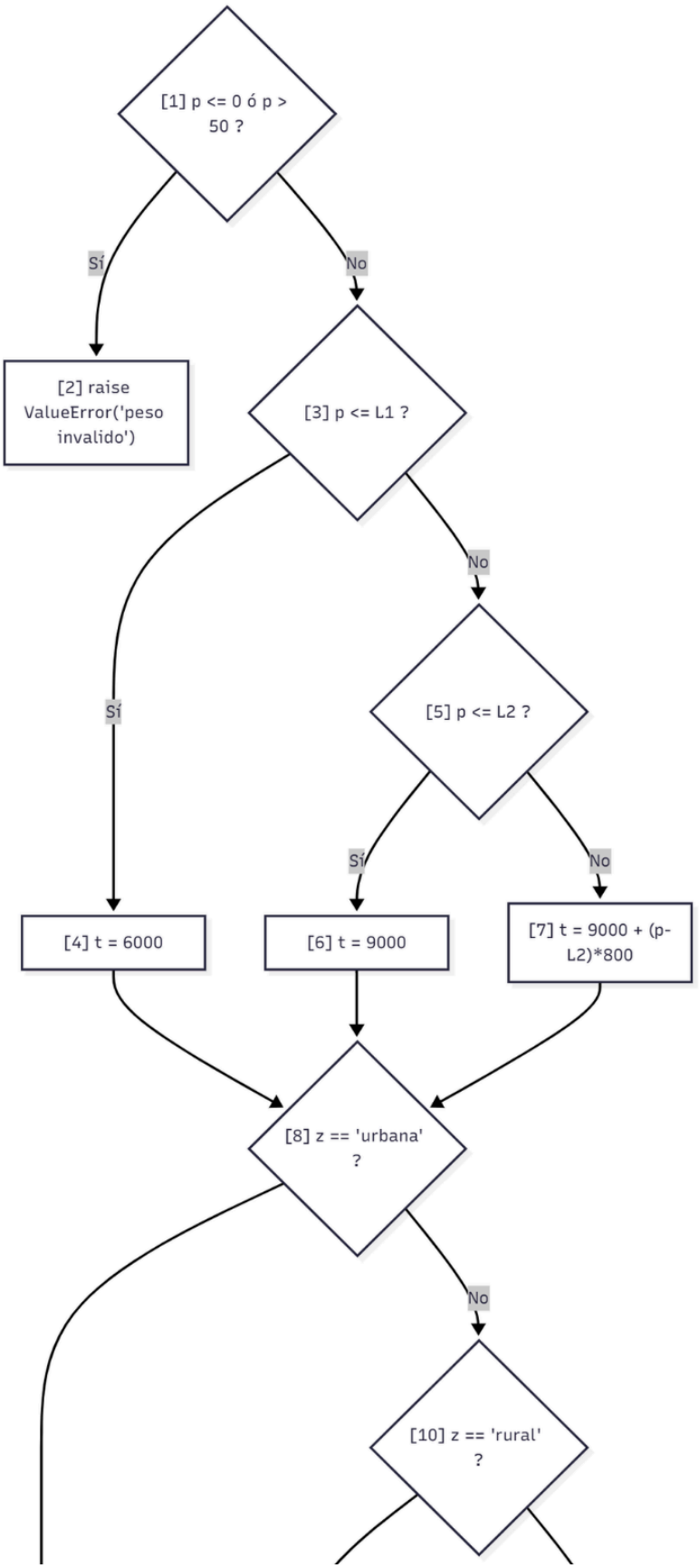
1. Base (peso 15 kg): $9 000 + $800 × 5 = $13 000
2. Zona Nacional (+45 %): $13 000 × 1.45 = $18 850
3. Fin de semana (+$2 000): $18 850 + $2 000 = $20 850
4. Descuento Cliente (Frecuente 5 %): $20 850 − (5 % × $20 850) = $19 807.5
5. Descuento Cupón (10 %): $19 807.5 − (10 % × $19 807.5) = $17 826.75
6. Redondeo: $17 826.75 → $17 800

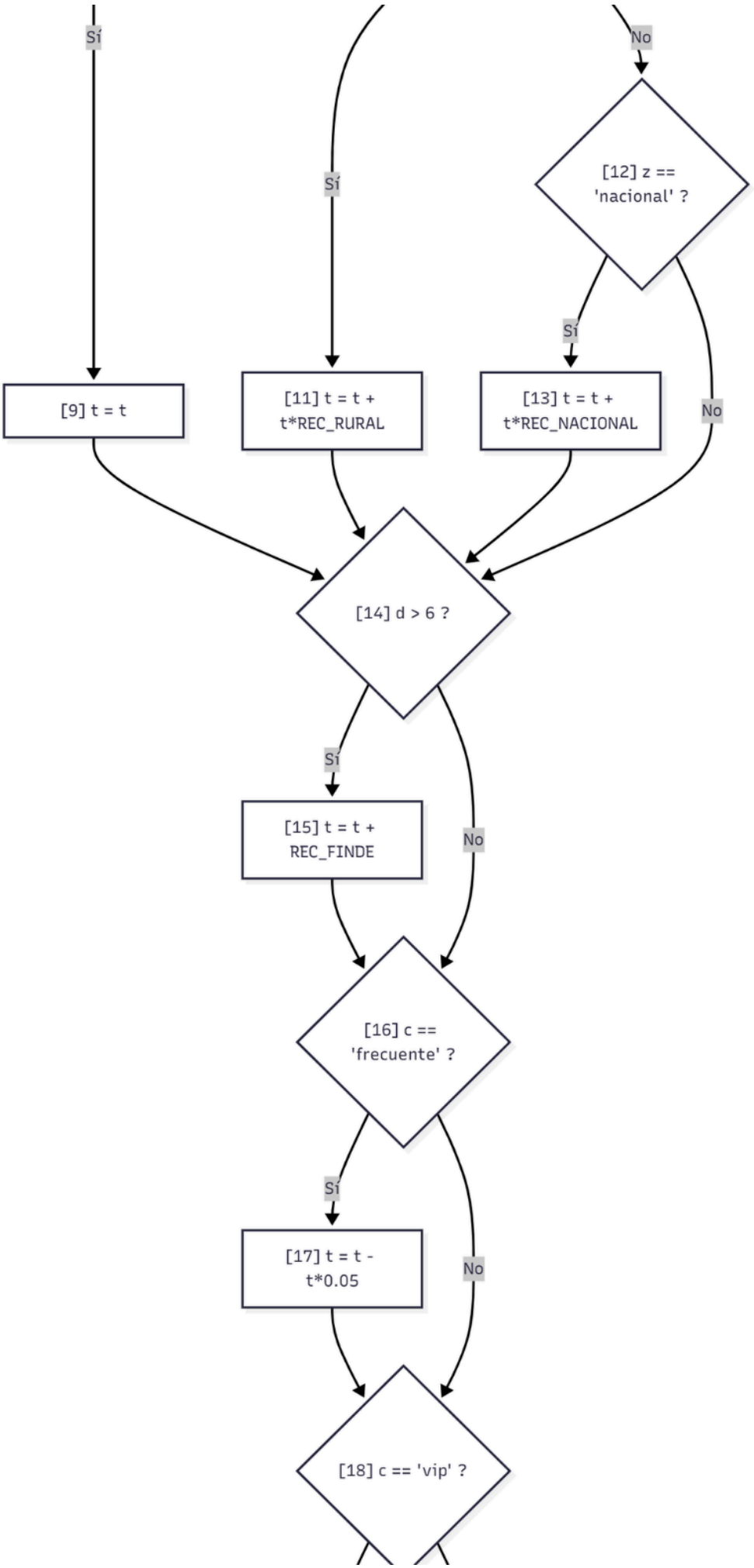
CP19

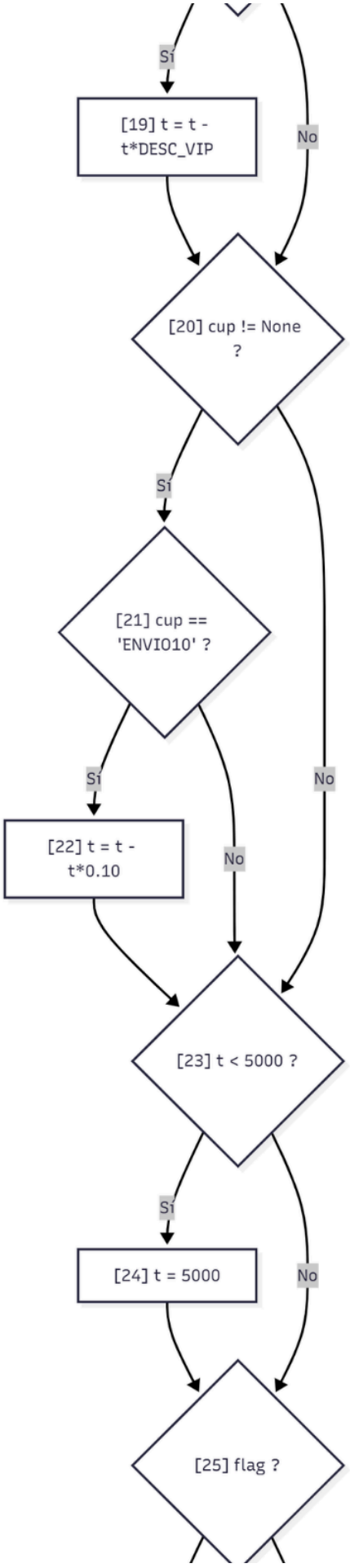
1. Base (peso 20 kg): $9 000 + $800 × 10 = $17 000
2. Zona Nacional (+45 %): $17 000 × 1.45 = $24 650
3. Fin de semana (+$2 000): $24 650 + $2 000 = $26 650
4. Sin descuentos (cliente normal, sin cupón) → subtotal $26 650
5. Redondeo: $26 650 → $26 700

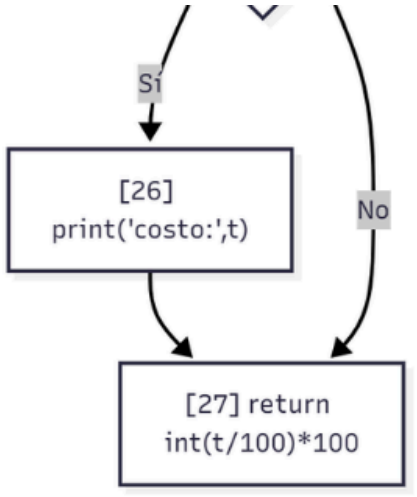
Diseño de casos de caja blanca

Grafo de flujo




Sí
Sí
No
[12] z == 'nacional' ?
Sí
[9] t = t
[11] t = t + t*REC_RURAL
[13] t = t + t*REC_NACIONAL
No
[14] d > 6 ?
Sí
[15] t = t + REC_FINDE
No
[16] c == 'frecuente' ?
Sí
[17] t = t - t*0.05
No
[18] c == 'vip' ?


Si
[19] t = t - t*DESC_VIP
No
[20] cup != None ?
Si
[21] cup == 'ENVIO10' ?
Si
[22] t = t - t*0.10
No
No
[23] t < 5000 ?
Si
[24] t = 5000
No
[25] flag ?



Complejidad ciclomática

Fórmula	**Cálculo**	**Resultado**
E – N + 2	40 aristas – 27 nodos + 2 = **15**	15
Nodos predicado + 1	13 decisiones (1,3,5,8,10,12,14,16,18,20,21,23,25) + 1 = **14**	14
Número de regiones (planar)	15 (contando la región exterior)	15

Caminos independientes

#	p (kg)	z (zona)	d (día)	c (cliente)	cup	flag	Ruta de nodos (resumen)
1	0	–	–	–	–	–	1 → 2 (peso inválido)
2	5	urbana	1	otro	None	False	1-3-4-8-9-14-16-18-20-23-25-27
3	5	rural	7	frecuente	None	False	1-3-4-8-10-11-14-15-16-17-18-20-23-25-27
4	12	nacional	1	vip	None	False	1-3-5-6-8-12-13-14-16-18-19-20-23-25-27
5	30	urbana	7	otro	None	False	1-3-7-8-9-14-15-16-18-20-23-25-27
6	30	rural	1	otro	None	False	1-3-7-8-10-11-14-16-18-20-23-25-27
7	30	nacional	1	otro	None	False	1-3-7-8-12-13-14-16-18-20-23-25-27

8	8	urbana	1	frecuente	None	True	1-3-4-8-9-14-16-17-18-20-23-25-26-27
9	8	urbana	1	vip	None	True	1-3-4-8-9-14-16-18-19-20-23-25-26-27
10	8	urbana	1	otro	ENVIO10	False	1-3-4-8-9-14-16-18-20-21-22-23-25-27
11	8	urbana	1	otro	"OTRO"	False	1-3-4-8-9-14-16-18-20-21-23-25-27
12	8	urbana	1	otro	None	True	1-3-4-8-9-14-16-18-20-23-25-26-27
13	8	urbana	7	frecuente	None	False	1-3-4-8-9-14-15-16-17-18-20-23-25-27
14	8	urbana	7	vip	None	False	1-3-4-8-9-14-15-16-18-19-20-23-25-27
15	8	urbana	7	otro	None	False	1-3-4-8-9-14-15-16-18-20-23-25-27

Cobertura de Ramas (Branch Coverage)

Al ejecutar el conjunto base de los 15 caminos independientes (casos 1 al 15 de la tabla anterior), se garantiza una cobertura de ramas del 100%. Esto se

debe a que la Complejidad Ciclomática V(G)=15 define exactamente el número de caminos necesarios para atravesar al menos una vez todas las aristas (bifurcaciones V/F) del grafo de flujo.

Cobertura de Sentencias (Statement Coverage)

Dado que la cobertura de ramas subsume (incluye) la cobertura de sentencias, ejecutar los 15 caminos independientes también asegura que el 100% de los nodos (líneas de código ejecutables) de la función `calc` hayan sido visitados al menos una vez.

Automatización, ejecución y defectos

Resultado de los test

```
PS C:\Users\Cristian\Documents\Documentos Universidad  2026-2\Ingenieria de Software\Parcial #3> cd Entregable
PS C:\Users\Cristian\Documents\Documentos Universidad  2026-2\Ingenieria de Software\Parcial #3\Entregable> pytest test_envios.py -v --cov=envios
============================================= test session starts =============================================
platform win32 -- Python 3.14.8, pytest-9.2.0, pluggy-1.7.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Cristian\Documents\Documentos Universidad  2026-2\Ingenieria de Software\Parcial #3\Entregable
plugins: anyio-4.15.1, platformdirs-4.12.4, cov-7.1.0
collected 20 items

test_envios.py::test_calc_casos_validos[CP01] PASSED                                                  [  5%]
test_envios.py::test_calc_casos_validos[CP02] FAILED                                                  [ 10%]
test_envios.py::test_calc_casos_validos[CP03] FAILED                                                  [ 15%]
test_envios.py::test_calc_casos_validos[CP07] FAILED                                                  [ 20%]
test_envios.py::test_calc_casos_validos[CP08] PASSED                                                  [ 25%]
test_envios.py::test_calc_casos_validos[CP09] PASSED                                                  [ 30%]
test_envios.py::test_calc_casos_validos[CP10] PASSED                                                  [ 35%]
test_envios.py::test_calc_casos_validos[CP11] FAILED                                                  [ 40%]
test_envios.py::test_calc_casos_validos[CP12] FAILED                                                  [ 45%]
test_envios.py::test_calc_casos_validos[CP13] FAILED                                                  [ 50%]
test_envios.py::test_calc_casos_validos[CP14] PASSED                                                  [ 55%]
test_envios.py::test_calc_casos_validos[CP15] PASSED                                                  [ 60%]
test_envios.py::test_calc_casos_validos[CP16] FAILED                                                  [ 65%]
test_envios.py::test_calc_casos_validos[CP17] FAILED                                                  [ 70%]
test_envios.py::test_calc_casos_validos[CP18] PASSED                                                  [ 75%]
test_envios.py::test_calc_casos_validos[CP19] FAILED                                                  [ 80%]
test_envios.py::test_calc_casos_invalidos[CP04] PASSED                                                [ 85%]
test_envios.py::test_calc_casos_invalidos[CP05] PASSED                                                [ 90%]
test_envios.py::test_calc_casos_invalidos[CP06] PASSED                                                [ 95%]
test_envios.py::test_calc_casos_invalidos[CP20] FAILED                                                [100%]
```

```
test_envios.py:61: Failed
---------- tests coverage ----------
_____ coverage: platform win32, python 3.14.8-final-0 _____

Name        Stmts   Miss  Cover
-----------------------------
envios.py      29      2    93%
-----------------------------
TOTAL          29      2    93%
========================================== short test summary info ==========================================
FAILED test_envios.py::test_calc_casos_validos[CP02] - AssertionError: Fallo en CP02 -> Esperado: $10200, Obtenido: $8400
FAILED test_envios.py::test_calc_casos_validos[CP03] - AssertionError: Fallo en CP03 -> Esperado: $14800, Obtenido: $14700
FAILED test_envios.py::test_calc_casos_validos[CP07] - AssertionError: Fallo en CP07 -> Esperado: $9000, Obtenido: $6000
FAILED test_envios.py::test_calc_casos_validos[CP16] - AssertionError: Fallo en CP16 -> Esperado: $17800, Obtenido: $16100
FAILED test_envios.py::test_calc_casos_validos[CP17] - AssertionError: Fallo en CP17 -> Esperado: $7700, Obtenido: $7600
FAILED test_envios.py::test_calc_casos_validos[CP19] - AssertionError: Fallo en CP19 -> Esperado: $26700, Obtenido: $24600
FAILED test_envios.py::test_calc_casos_invalidos[CP20] - Failed: DID NOT RAISE any of (ValueError, Exception)
======================================== 10 failed, 10 passed in 0.21s ========================================
PS C:\Users\Cristian\Documents\Documentos Universidad  2026-2\Ingenieria de Software\Parcial #3\Entregable>
```

Reportes de defectos

ID Defecto	Título	Requisito Incumplido	Pasos (Resumen de entradas)	Resultado Esperado	Resultado Obtenido (Código base)	Severidad / Prioridad	Justificación
DEF-01	Inclusión errónea del límite L1 en Tramo 1	Regla D=1: Tramo 1 es estrictamente p < 3. Si p == 3, pertenece al Tramo 2 (base $9.000).	Ejecutar calc(p=3, z="urbana", c="normal", d=1) (CP07).	Tarifa base = $9.000	Tarifa base = $6.000 (p <= CONFIG["L1"])	Alta / Alta	Genera una pérdida financiera directa del 33.3% sobre la tarifa base en el valor frontera exacto de 3 kg.
DEF-02	Omisión del recargo de fin de semana para el día sábado (día 6)	Regla Fin de Semana: Días 6 y 7 aplican recargo de $2.000 antes de descuentos.	Ejecutar calc(p=5, z="rural", c="frecuente", d=6, cup="ENVIO10") (CP02) o calc(p=20, z="nacional", c="normal", d=6) (CP19).	Sumar recargo de $2.000 (CP02: $10.200; CP19: $26.700).	No aplica recargo por condición d > 6 (CP02: $8.400; CP19: $24.600).	Alta / Alta	Invalida el cobro de recargo en el 50% de los días del fin de semana (sábados), causando fuga masiva de ingresos.
DEF-03	Truncamiento en vez de redondeo comercial al múltiplo de $100	Regla de Redondeo: Redondear al múltiplo de 100 $más\ cercano($ 50 sube al siguiente 100).	Ejecutar calc(p=12, z="nacional", c="vip", d=7) (CP03) o calc(p=4, z="urbana", c="vip", d=5) (CP17).	Redondeo superior (CP03: subtotal $14.764, 5 → $14.800; CP17: subtotal $7.650 → $7.700).	Truncamiento inferior vía int(t / 100) * 100 (CP03: $14.700; CP17: $7.600).	Media / Media	Genera descuadres y diferencias sistemáticas de $100 en la facturación de envíos con fracciones de centenas.

DEF-04	Ausencia de validación para día de la semana fuera de rango	Regla Días: Parámetro dia debe ser un entero entre 1 y 7; valores externos deben levantar excepción.	Ejecutar calc(p=9, z="urbana", c="normal", d=8) (CP20).	Lanzar excepción ValueError ("dia invalido").	Retorna valor numérico ($11.000) y activa erróneamente recargo por d > 6.	Media / Media	Acepta datos corruptos o fuera de especificación sin alertar al invocador ni controlar la integridad de entrada.
DEF-05	Ausencia de validación para zonas geográficas no permitidas	Regla Zonas: Las únicas zonas permitidas son "urbana", "rural" y "nacional". Otras opciones deben lanzar excepción.	Ejecutar calc(p=5, z="internacional", c="normal", d=2).	Lanzar excepción ValueError ("zona invalida").	Retorna tarifa base sin recargo ($9.000) al ignorar silenciosamente los bloques elif.	Media / Media	Permite despachos a zonas no soportadas sin tarifar recargos operativos ni levantar error.

Métricas de Calidad

Porcentaje de Casos que Pasan vs. que Fallan

- Total de casos ejecutados: 20 casos
- Casos que pasan (PASS): 10 casos (CP01, CP04, CP05, CP06, CP08, CP09, CP10, CP14, CP15, CP18) →\rightarrow→ 50.0%
- Casos que fallan (FAIL): 10 casos (CP02, CP03, CP07, CP11, CP12, CP13, CP16, CP17, CP19, CP20) →\rightarrow→ 50.0%

Defectos por Severidad

- Severidad Alta: 2 defectos (DEF-01, DEF-02) →\rightarrow→ 40.0%
- Severidad Media: 3 defectos (DEF-03, DEF-04, DEF-05) → 60.0%
- Severidad Baja: 0 defectos → 0.0%
- Total Defectos Detectados: 5 defectos

Mantenimiento y refactorización

Identificación de malos olores (code smells)

#	Nombre técnico	Línea / sección	Deuda técnica que genera
1	Flag Argument	L10 y L37-38 (flag, print)	La función hace dos cosas: calcula e imprime. Rompe SRP y mezcla E/S con lógica de negocio. Es difícil de probar.
2	Poor Naming	L10 (p, z, c, d, cup, t)	Nadie entiende las variables sin leer todo el cuerpo. Aumenta el costo de mantenimiento y el riesgo de errores.
3	Magic Numbers	L14, 16, 18, 29, 34, 36, 50 (6000, 9000, 800, 0.05, 0.10, 5000, 100, 50)	Las tarifas y porcentajes están ocultos en el código. Un cambio de tarifa obliga a buscarlos uno por uno.
4	Magic Strings	L19-33 ("urbana", "vip", "ENVIO10"…)	Un error de tipeo falla en silencio. No hay catálogo único de valores válidos.
5	Long Method / Multiple Responsibilities	L10-39	Una sola función valida, calcula la tarifa, aplica recargos y descuentos, redondea e imprime. Rompe SRP.

6	Switch Statements (cadenas if/elif)	L19-24 y L28-34	Cada zona, cliente o cupón nuevo obliga a modificar la función. Rompe OCP. Es la causa directa del patrón Estrategia.
7	Dead / Redundant Code	L20 (t = t), L32 (cup != None)	Ruido que confunde. != None debería ser is not None.
8	Missing Validation / Primitive Obsession	L11-12 y ausencia de chequeos de z, d, c	Las entradas inválidas pasan sin error (DEF-04 y DEF-05). Los strings y enteros sueltos hacen de dominio.
9	Inconsistent Config Usage	L2-9 (CONFIG mezcla valores usados y valores literales)	Una parte de la configuración está en CONFIG y otra está en el código. No hay una única fuente de verdad.
10	Truncation en lugar de redondeo / Lógica de bajo nivel inline	L39 (int(t/100)*100)	Es la causa de DEF-03. Los float acumulan error y la lógica de redondeo no es reutilizable.

Clasificación de Mantenimiento

Tipo	Cambios	Justificación
Correctivo	DEF-01 (p < L1), DEF-02 (d >= 6), DEF-03 (redondeo half-up), DEF-04/05 (validar día, zona y cliente)	Arreglan fallos que ya están reportados y que dan resultados incorrectos.

Adaptativo	Eliminar flag (breaking change) y mantener el alias calc. Aplicar las reglas D=1, E=0, B=1 y F=4. Usar Decimal en vez de float.	Adapta el código a reglas de negocio y a una interfaz que cambiaron.
Perfectivo	Nombres descriptivos, constantes, métodos de apoyo y docstrings	Mejora la legibilidad y la estructura sin cambiar el comportamiento.
Preventivo	Patrón Estrategia (ABC) y diccionarios de estrategias. Validadores aislados.	Reduce el riesgo de futuros defectos y permite agregar descuentos sin tocar el cálculo (OCP).

Código refactorizado

```
# envios.py
"""Cálculo de costo de envío de EnvíaYa (v2.0.0, refactorizado).

Patrón Estrategia aplicado a los DESCUENTOS (cliente y cupón).
Reglas de negocio: D=1, E=0, B=1, F=4.
"""
from abc import ABC, abstractmethod
from decimal import Decimal, ROUND_HALF_UP

#
----------------------------------------------------------------
--------
# Constantes de negocio
#
----------------------------------------------------------------
--------
LIMITE_TRAMO_1 = Decimal("3")        # kg (Regla D=1: el límite NO
pertenece al tramo 1)
LIMITE_TRAMO_2 = Decimal("10")       # kg (el límite SÍ pertenece
al tramo 2)
PESO_MAXIMO = Decimal("50")          # kg

TARIFA_TRAMO_1 = Decimal("6000")
TARIFA_TRAMO_2 = Decimal("9000")
COSTO_KG_ADICIONAL = Decimal("800")  # por kg sobre LIMITE_TRAMO_2

RECARGO_RURAL = Decimal("0.10")
RECARGO_NACIONAL = Decimal("0.45")
```

```
RECARGO_FINDE = Decimal("2000")      # Regla E=0: se suma ANTES de
descuentos

DESCUENTO_FRECUENTE = Decimal("0.05")
DESCUENTO_VIP = Decimal("0.15")
DESCUENTO_CUPON_ENVIO10 = Decimal("0.10")

COSTO_MINIMO = Decimal("5000")
MULTIPLO_REDONDEO = Decimal("100")

DIA_SABADO = 6
DIA_DOMINGO = 7
DIA_MINIMO = 1
DIA_MAXIMO = 7

ZONA_URBANA = "urbana"
ZONA_RURAL = "rural"
ZONA_NACIONAL = "nacional"
RECARGOS_POR_ZONA = {
    ZONA_URBANA: Decimal("0"),
    ZONA_RURAL: RECARGO_RURAL,
    ZONA_NACIONAL: RECARGO_NACIONAL,
}

# Compatibilidad con el banco de pruebas existente
CONFIG = {
    "L1": int(LIMITE_TRAMO_1),
    "L2": int(LIMITE_TRAMO_2),
    "RECARGO_RURAL": float(RECARGO_RURAL),
    "RECARGO_NACIONAL": float(RECARGO_NACIONAL),
    "DESC_VIP": float(DESCUENTO_VIP),
    "RECARGO_FINDE": int(RECARGO_FINDE),
}


#
------------------------------------------------------------------
--------
# Patrón Estrategia: descuentos
#
------------------------------------------------------------------
--------
class EstrategiaDescuento(ABC):
    """Interfaz común de todas las estrategias de descuento."""

    @abstractmethod
    def aplicar(self, monto: Decimal) -> Decimal:
```

```
        """Devuelve el monto resultante tras aplicar el
descuento."""


class DescuentoPorcentual(EstrategiaDescuento):
    """Base reutilizable para descuentos expresados como
porcentaje."""

    def __init__(self, porcentaje: Decimal):
        self._porcentaje = porcentaje

    def aplicar(self, monto: Decimal) -> Decimal:
        return monto - monto * self._porcentaje


class SinDescuento(EstrategiaDescuento):
    def aplicar(self, monto: Decimal) -> Decimal:
        return monto


class DescuentoClienteFrecuente(DescuentoPorcentual):
    def __init__(self):
        super().__init__(DESCUENTO_FRECUENTE)


class DescuentoClienteVip(DescuentoPorcentual):
    def __init__(self):
        super().__init__(DESCUENTO_VIP)


class DescuentoCuponEnvio10(DescuentoPorcentual):
    def __init__(self):
        super().__init__(DESCUENTO_CUPON_ENVIO10)


ESTRATEGIAS_CLIENTE = {
    "normal": SinDescuento(),
    "frecuente": DescuentoClienteFrecuente(),
    "vip": DescuentoClienteVip(),
}

ESTRATEGIAS_CUPON = {
    "ENVIO10": DescuentoCuponEnvio10(),
}
# Regla F=4: un cupón inexistente se ignora.
ESTRATEGIA_CUPON_POR_DEFECTO = SinDescuento()
```

```
#
--------------------------------------------------------------------
--------
# Validaciones (DEF-04 / DEF-05)
#
--------------------------------------------------------------------
--------
def _validar_peso(peso) -> Decimal:
    if isinstance(peso, bool) or not isinstance(peso, (int, float,
Decimal)):
        raise ValueError("peso invalido: debe ser numerico")
    peso_decimal = Decimal(str(peso))
    if not peso_decimal.is_finite() or peso_decimal <= 0 or
peso_decimal > PESO_MAXIMO:
        raise ValueError(f"peso invalido: debe estar en (0,
{PESO_MAXIMO}]")
    return peso_decimal


def _validar_zona(zona) -> str:
    if zona not in RECARGOS_POR_ZONA:
        raise ValueError(f"zona invalida: {zona!r}")
    return zona


def _validar_cliente(tipo_cliente) -> str:
    if tipo_cliente not in ESTRATEGIAS_CLIENTE:
        raise ValueError(f"tipo de cliente invalido:
{tipo_cliente!r}")
    return tipo_cliente


def _validar_dia(dia) -> int:
    if isinstance(dia, bool) or not isinstance(dia, int) or not
DIA_MINIMO <= dia <= DIA_MAXIMO:
        raise ValueError(f"dia invalido: debe ser entero entre
{DIA_MINIMO} y {DIA_MAXIMO}")
    return dia


#
--------------------------------------------------------------------
--------
# Métodos de apoyo
#
--------------------------------------------------------------------
--------
def _calcular_tarifa_base(peso: Decimal) -> Decimal:
```

```
    """Tarifa por tramos de peso (Regla D=1: tramo 1 es peso < L1)
(DEF-01)."""
    if peso < LIMITE_TRAMO_1:
        return TARIFA_TRAMO_1
    if peso <= LIMITE_TRAMO_2:
        return TARIFA_TRAMO_2
    return TARIFA_TRAMO_2 + (peso - LIMITE_TRAMO_2) *
COSTO_KG_ADICIONAL


def _aplicar_recargo_zona(monto: Decimal, zona: str) -> Decimal:
    return monto + monto * RECARGOS_POR_ZONA[zona]


def _es_fin_de_semana(dia: int) -> bool:
    """Sábado (6) y domingo (7) (DEF-02)."""
    return dia >= DIA_SABADO


def _obtener_estrategia_cupon(cupon) -> EstrategiaDescuento:
    return ESTRATEGIAS_CUPON.get(cupon,
ESTRATEGIA_CUPON_POR_DEFECTO)


def _redondear_comercial(monto: Decimal) -> int:
    """Redondea al múltiplo de 100 más cercano; la mitad sube
(DEF-03)."""
    cientos = (monto / MULTIPLO_REDONDEO).quantize(Decimal("1"),
rounding=ROUND_HALF_UP)
    return int(cientos * MULTIPLO_REDONDEO)


#
----------------------------------------------------------------
--------
# API pública
#
----------------------------------------------------------------
--------
def calcular_costo_envio(peso, zona, tipo_cliente, dia,
cupon=None) -> int:
    """Calcula el costo final del envío en pesos (múltiplo de
100).

    Args:
        peso: kg, 0 < peso <= 50.
        zona: "urbana" | "rural" | "nacional".
        tipo_cliente: "normal" | "frecuente" | "vip".
```

```
        dia: entero 1 (lunes) a 7 (domingo).
        cupon: código opcional; si no existe se ignora.

    Raises:
        ValueError: si peso, zona, tipo_cliente o dia son
inválidos.
    """
    peso_valido = _validar_peso(peso)
    zona_valida = _validar_zona(zona)
    cliente_valido = _validar_cliente(tipo_cliente)
    dia_valido = _validar_dia(dia)

    costo = _calcular_tarifa_base(peso_valido)
    costo = _aplicar_recargo_zona(costo, zona_valida)
    if _es_fin_de_semana(dia_valido):
        costo += RECARGO_FINDE

    # Regla B=1: se aplican ambos descuentos (cliente y cupón)
    costo = ESTRATEGIAS_CLIENTE[cliente_valido].aplicar(costo)
    costo = _obtener_estrategia_cupon(cupon).aplicar(costo)

    costo = max(costo, COSTO_MINIMO)
    return _redondear_comercial(costo)


def calc(p, z, c, d, cup=None):
    """Alias de compatibilidad (sin el argumento flag,
eliminado)."""
    return calcular_costo_envio(p, z, c, d, cup)
```